

RANDOMIZED BLOCK LOW-RANK MATRIX COMPRESSION BY TAGGING*

KATHERINE J. PEARCE[†], ANNA YESYPENKO[‡], JAMES LEVITT[‡], AND PER-GUNNAR
MARTINSSON[†]

Abstract. In this work, we present randomized compression algorithms for flat rank-structured matrices with shared bases, termed uniform Block Low-Rank (BLR) matrices. Our main contribution is a technique called tagging, which improves upon the efficiency of existing algorithms for basis matrix computation while preserving accuracy. Tagging operates on the matrix using matrix-vector products of the matrix and its adjoint, making it suitable for black-box environments where accessing individual matrix entries is computationally expensive or infeasible. We show tagging requires a constant number of matrix-vector products coupled with linear post-processing; crucially, the asymptotic pre-factors in tagging depend only on the rank parameter and the underlying problem geometry. We also establish a theoretical connection between the optimal construction of tagging matrices and projective varieties in algebraic geometry, suggesting a hybrid numeric-symbolic avenue of future work. To validate our approach, we apply tagging to compress uniform BLR matrices arising from the discretization of integral and partial differential equations. Empirical results show that tagging outperforms alternative compression techniques, significantly reducing both the number of required matrix-vector products and overall computational time. These findings highlight the practicality and scalability of tagging as an efficient method for flat rank-structured matrices in scientific computing.

Key words. Randomized numerical linear algebra, rank-structured matrix, black-box algorithms, block low-rank matrix

MSC codes. 68Q25, 65F99, 68W20

1. Introduction. In scientific computing and data science, many applications involve matrices that are dense but “data-sparse,” admitting certain low-rank approximations that compress the matrices while preserving critical information. Algorithms to compress data-sparse matrices can achieve better performance by invoking *rank structure*, where the input matrices are tessellated into blocks that are either small enough in size to apply dense algorithms or are of low numerical rank.

Many rank-structured matrix formats have been successfully utilized in engineering and data science applications. These formats are typically classified as either hierarchical [7, 8, 10, 18, 21, 24] or flat [2, 3, 5, 30, 33, 46]; illustrative examples of hierarchical vs. flat rank-structure formats are shown in Fig. 1. Both formats can be further characterized by either a weak or strong admissibility criterion. In weakly admissible formats (as in Fig. 1), every off-diagonal block is admissible, or treated as low-rank, whereas admissible blocks of strongly admissible formats (cf. Fig. 2) correspond to “far-field” interactions (as in, e.g., the Fast Multipole Method [22]).

The last defining feature of rank-structured matrices most typically pertains to hierarchical formats, namely whether all admissible blocks in the same block-row or block-column are well-approximated by the same low-rank basis matrices. Frequently, basis matrix computations can be accelerated without impacting the accuracy of the low-rank approximation by invoking a shared basis assumption, namely that the row or column spaces of all admissible blocks within the same block-row or block-column

*Submitted to the editors May 7, 2025.

Funding: This work was funded by the National Science Foundation (DMS-2401889, DMS-1952735, DMS-2313434), Office of Naval Research (N00014-18-1-2354), and the Department of Energy (DE-SC0022251).

[†]Department of Mathematics & Oden Institute, University of Texas at Austin (katherine.pearce@austin.utexas.edu, pgm@oden.utexas.edu).

[‡]Oden Institute, University of Texas at Austin (annayesy@utexas.edu, jlevitt@utexas.edu).

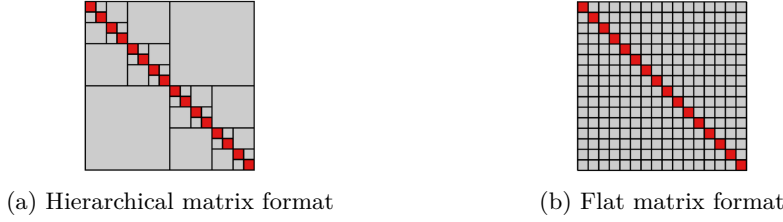


Fig. 1: An illustration of hierarchical vs. flat rank-structured matrix formats. In (a), the rank-structure format uses a hierarchy of tessellations of the underlying problem domain. In (b), the flat format corresponds to a single tessellation of the domain. Hierarchical formats can achieve better asymptotic (e.g. linear) complexity than flat formats, but at the expense of more complicated implementations. By contrast, flat formats are sufficient in many applications, and working with them is much simpler.

are spanned by the same basis. A shared basis assumption is utilized in the strongly admissible uniform $\mathcal{H}^1$ -matrix [24, 38] and $\mathcal{H}^2$ -matrix [16, 26] formats, while the HSS [10, 54] and HBS [18, 36] formats use shared bases under weak admissibility.

In this manuscript, we focus on a flat format termed block low-rank (BLR) [2, 3, 4, 30], more specifically on BLR matrices under a shared basis assumption, which we refer to as uniform BLR matrices throughout this work. Though the BLR format has enjoyed much practical success and performance optimization [1, 9, 32, 50, 52], its utility in many applications has not yet been explored, particularly those in which the matrix entries cannot be directly accessed. Rather, in these black-box problems, the input matrix is assumed to be accessible only through a fast method to evaluate matrix-vector products, rather than through direct access to its entries. Here, the goal is often matrix “reconstruction” in terms of low-rank basis matrices, enabling downstream operations (e.g., inversion or LU factorization) and simplifying formation of matrix products. These compressed matrix representations have broad applicability in scientific computing, for instance in deriving rank-structured representations of integral operators [41, 57] or accelerating sparse direct solvers [44, 56].

Randomized algorithms have been shown to be very effective in black-box problem environments, particularly *randomized sketching*, in which the row and column spaces of an input matrix are approximated by analyzing how the matrix and its transpose act on tall thin matrices drawn from random matrix distributions [27, 36, 38, 40]. Namely, suppose that $\mathbf{A} \in \mathbb{R}^{N \times N}$ has some given rank structure where the numerical ranks of admissible blocks are upper bounded by $k \ll N$, but that $\mathbf{A}$ is only accessible through some fast black-box algorithm. In other words, given tall thin $\mathbf{\Omega}, \mathbf{\Psi} \in \mathbb{R}^{N \times r}$, $r = \mathcal{O}(k)$, we can quickly evaluate $\mathbf{Y} = \mathbf{A}\mathbf{\Omega}$ and $\mathbf{Z} = \mathbf{A}^*\mathbf{\Psi}$. Our goal is then to reconstruct $\mathbf{A}$ as efficiently as possible, using only the information in the set $\{\mathbf{Y}, \mathbf{\Omega}, \mathbf{Z}, \mathbf{\Psi}\}$ [38, 42, 43].

Levitt and Martinsson [36] introduced the first fully black-box linear-complexity randomized algorithm to compress (weakly admissible) HBS matrices, inspired by the linear-complexity HBS compression algorithm of [42]. Recent work [58] proposed a fully black-box algorithm for the simultaneous compression and LU factorization of general $\mathcal{H}^2$ matrices under strong admissibility, also achieving linear complexity. The present work focuses on the efficient black-box compression of uniform BLR matrices. Although flat formats have higher asymptotic complexity compared to hierarchical formats, they often feature modest constant factors in storage and can yield significant

practical speed-ups.

We first adapt the block nullification algorithm of [36, 58] for strongly admissible uniform BLR matrices. We then introduce a new randomized compression algorithm for uniform BLR matrices based on a method we refer to as *tagging*, in which null spaces of small submatrices are computed to exclude contributions from inadmissible blocks in the random sketches $\mathbf{Y}$ and $\mathbf{Z}$. As in [36, 58], the strategic exclusion of inadmissible blocks permits the same sample matrices $\mathbf{Y}$ and $\mathbf{Z}$ to be used to compress every admissible block-row and block-column of the input matrix, which can be straightforwardly parallelized for optimized performance. Additionally, our tagging method has a significantly smaller asymptotic pre-factor, and more critically, its sampling complexity for basis matrix computation is constant—independent of the problem size even in flat formats. We show that tagging improves on the performance of block nullification while maintaining approximation accuracy.

Contributions. We propose two new randomized compression schemes for flat rank-structured matrices with shared bases, termed uniform BLR. The first is based on our extension of the block nullification method in [36, 58] to uniform BLR matrices under a strong admissibility criterion. The second is our main contribution, the novel method of tagging to compress uniform BLR matrices. We draw a connection between the tagging matrices utilized in our method and Plücker coordinates in projective space, before presenting a highly efficient method of generating tagging matrices that works well in practice. We analyze the asymptotic complexities of both schemes and compare their performance in compressing matrices arising from discretizations of integral and partial differential equations, demonstrating the superior computational efficiency of tagging.

Outline. Our manuscript is structured as follows. Section 2 covers necessary linear algebra preliminaries and details the uniform BLR matrix format. Section 3 illustrates our modification of the block nullification for basis matrix computations and analyzes its asymptotic complexity, and Section 4 introduces the new method of tagging with detailed complexity analysis. Section 5 outlines the relationship between tagging and projective varieties and provides practical strategies that yield high-quality random sketches with tagging. Section 6 finishes the compression procedure for uniform BLR matrices before showcasing numerical results in Section 7.

2. Preliminaries. We briefly summarize necessary background for our work, following the presentation of [43]. We then outline the BLR matrix format as in [5].

2.1. Notation. A vector $\mathbf{x} \in \mathbb{R}^n$ is measured by the Euclidean norm $\|\mathbf{x}\| = (\sum_i |x_i|^2)^{\frac{1}{2}}$, and a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is equipped with the corresponding operator norm $\|\mathbf{A}\| = \sup_{\|\mathbf{x}\|=1} \|\mathbf{A}\mathbf{x}\|$. We let $[m]$ denote the integers $1, 2, \dots, m$. We adopt the notation of Golub and Van Loan [20] to reference submatrices; namely, if $\mathbf{A}$ is an $m \times n$ matrix, and $I = [i_1, i_2, \dots, i_k] \subset [m]$ and $J = [j_1, j_2, \dots, j_l] \subset [n]$ are (row and column, resp.) index sets, then $\mathbf{A}(I, J)$ denotes the $k \times l$ matrix

$$\mathbf{A}(I, J) = \begin{bmatrix} \mathbf{A}(i_1, j_1) & \mathbf{A}(i_1, j_2) & \dots & \mathbf{A}(i_1, j_l) \\ \mathbf{A}(i_2, j_1) & \mathbf{A}(i_2, j_2) & \dots & \mathbf{A}(i_2, j_l) \\ \vdots & \vdots & & \vdots \\ \mathbf{A}(i_k, j_1) & \mathbf{A}(i_k, j_2) & \dots & \mathbf{A}(i_k, j_l) \end{bmatrix}.$$

The abbreviation $\mathbf{A}(I, :)$ designates the submatrix $\mathbf{A}(I, [n])$, and $\mathbf{A}(:, J)$ is defined analogously. The transpose of $\mathbf{A}$ is given by $\mathbf{A}^*$, and a matrix $\mathbf{U}$ is said to be orthonormal if its columns are orthonormal, $\mathbf{U}^* \mathbf{U} = \mathbf{I}$.

2.2. The QR factorization. Every $m \times n$ matrix $\mathbf{B}$ has a (full) QR factorization

$$(2.1) \quad \begin{matrix} \mathbf{B} \\ m \times n \end{matrix} = \begin{matrix} \mathbf{Q} \\ m \times m \end{matrix} \begin{matrix} \mathbf{R} \\ m \times n \end{matrix},$$

where $\mathbf{Q}$ is orthonormal and $\mathbf{R}$ is upper-triangular. If $\mathbf{B}$ has rank at most k , then it has a rank- k partial QR factorization given by

$$(2.2) \quad \begin{matrix} \mathbf{B}_k \\ m \times n \end{matrix} = \begin{matrix} \mathbf{Q}_k \\ m \times k \end{matrix} \begin{matrix} \mathbf{R}_k \\ k \times n \end{matrix},$$

where $\mathbf{Q}_k$ is orthonormal and $\mathbf{R}_k$ is upper-triangular.

2.3. Functions for orthonormal bases. For matrix $\mathbf{B}$ of rank $\leq k$, the function outputting $\mathbf{Q}$ with k orthonormal columns spanning the column space of $\mathbf{B}$ is

$$\mathbf{Q} = \text{col}(\mathbf{B}, k).$$

Analogously, for matrix $\mathbf{B}$ with null space of dimension $\geq k$, we denote a function that returns a matrix $\mathbf{Z}$ with k orthonormal columns in the null space of $\mathbf{B}$ by

$$\mathbf{Z} = \text{null}(\mathbf{B}, k).$$

To implement `col` and `null`, a rank-revealing factorization, e.g., the SVD or column-pivoted QR, is usually necessary. However, for the particular input matrices $\mathbf{B}$ arising in the present context, unpivoted QR can safely be used. (To be precise, the input matrices will be “sample matrices” in a randomized embedding, cf. Section 2.4.)

2.4. Randomized range-finding. Suppose $m \times n$ matrix $\mathbf{B}$ can be accurately approximated by a rank- k matrix, and we seek a matrix whose columns form an approximate orthonormal basis (ON-basis) for the column space of $\mathbf{B}$. Often referred to as range-finding, we seek an orthonormal matrix $\mathbf{Q}$ such that $\|\mathbf{B} - \mathbf{Q}\mathbf{Q}^*\mathbf{B}\|$ is small. This problem can be addressed efficiently by *randomized sketching*, where the column space of $\mathbf{B}$ is approximated by analyzing how $\mathbf{B}$ acts on matrices drawn from random distributions [27, 34, 53]. In general, randomized range-finding proceeds as follows:

1. Choose a small integer p for how much “oversampling” is done (e.g. $p = 10$).
2. Draw an $n \times (k + p)$ random matrix $\mathbf{G}$ (e.g. Gaussian [27, 34]).
3. Form the $m \times (k + p)$ random sketch $\mathbf{Y} = \mathbf{B}\mathbf{G}$.
4. Compute $\mathbf{Q} = \text{col}(\mathbf{Y}, k)$.

Each column of $\mathbf{Y}$ is then a random linear combination of the columns of $\mathbf{B}$, and the probability of obtaining an accurate column space approximation of $\mathbf{B}$ with the column space of $\mathbf{Y}$ approaches 1 rapidly as p increases; notably, this probability depends only on p (not on m or n , or any other properties of $\mathbf{B}$); cf. [27] and [45, Section 11].

2.5. Block Low-Rank (BLR) Matrix Compression. We focus on the randomized compression of $N \times N$ matrices $\mathbf{A}$ admitting a *block low-rank (BLR)* format. BLR matrices are tessellated into b row and b column blocks according to a flat (vs. hierarchical) rank structure. An example of a BLR matrix is illustrated in Fig. 2.

Usually, strong admissibility is assumed for BLR matrices, as opposed to weak admissibility where every off-diagonal block is treated as low-rank (Fig. 1(b)), since the asymptotic complexity is the same as weak admissibility in flat formats [5]. Under a strong admissibility condition, the matrix blocks that correspond to the neighbors

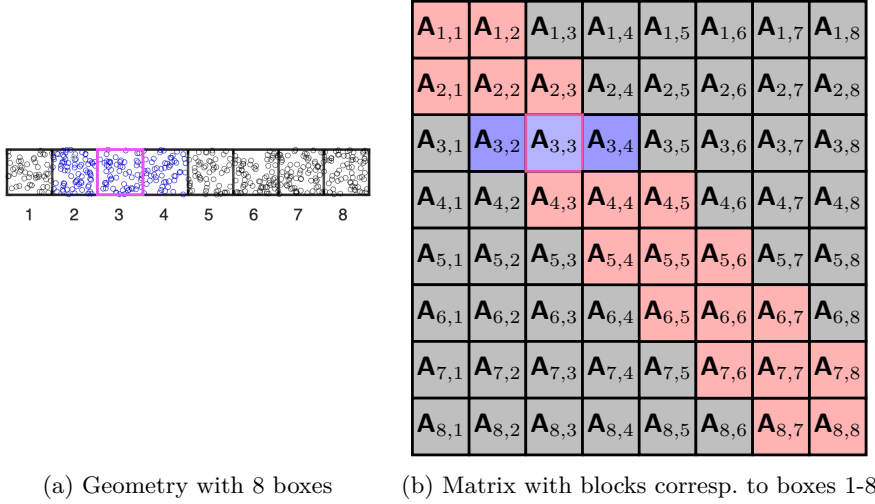


Fig. 2: Strongly admissible BLR matrix $\mathbf{A}$ in (b) corresponding to the quasi-1D geometry in (a). We denote the points in this geometry by $\{x_i\}_{i=1}^N$, and I_σ are the indices of points in box $\sigma = 1, \dots, b$ (here, $b = 8$). In matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$, we define sub-blocks $\mathbf{A}_{\sigma,\tau} = \mathbf{A}(I_\sigma, I_\tau)$, whose entries are given by some function $\mathcal{K}$ on $\{x_i\}_{i=1}^N$, so that $\mathbf{A}(i, j) = \mathcal{K}(x_i, x_j)$. Admissible blocks treated as low-rank are shown in gray.

of a given box are also treated densely, not as low-rank. To determine the neighbors of a given block, the problem geometry is subdivided until each box (corresponding to a matrix block) reaches the desired cardinality [15, 17, 31, 48]. The neighbors of a given matrix block are the blocks whose corresponding boxes share a boundary with the given box, including itself, for a total of up to 3^d neighbors in a d -dimensional geometry. For example, in Fig. 2, the neighbor set of box 3 (outlined in magenta in the quasi-1D geometry) is given by $\mathcal{N}_3 = \{2, 3, 4\}$ (in blue). The reader is referred to [7, 25, 41, 46] for more detailed treatments of rank-structured matrix admissibility.

Analogous to the uniform $\mathcal{H}^1$ rank-structured matrix format [24, 37, 38], we define a uniform BLR matrix $\mathbf{A}$ as a BLR matrix for which low-rank blocks within the same block-row or block-column share the same bases of their row or column spaces. Fig. 3 illustrates this with $\mathbf{A}$ from Fig. 2. To compress (strongly admissible) uniform BLR matrices, we compute basis matrices $\mathbf{U}_\ell$ and $\mathbf{V}_m$ for low-rank block $\mathbf{A}_{\ell,m}$ such that

$$(2.2) \quad \underbrace{\mathbf{A}_{\ell,m}}_{m \times m} = \underbrace{\mathbf{U}_\ell}_{m \times k} \underbrace{\tilde{\mathbf{A}}_{\ell,m}}_{k \times k} \underbrace{\mathbf{V}_m^*}_{k \times m},$$

where

$$(2.3) \quad \begin{aligned} \mathbf{U}_\ell &= \text{col}(\mathbf{A}(I_\ell, I_{i \in \mathcal{F}_\ell}), k), \\ \mathbf{V}_m &= \text{col}(\mathbf{A}^*(I_{i \in \mathcal{F}_m}, I_m), k). \end{aligned}$$

Here, if $\mathcal{N}_\ell$ denotes the set of neighbors of box ℓ , let $\mathcal{F}_\ell = [b] \setminus \mathcal{N}_\ell$ denote the set complement of $\mathcal{N}_\ell$, and similarly for $\mathcal{F}_m$. The shared bases assumption used in (2.3) reduces the number of basis matrices from $\mathcal{O}(b^2)$ to $\mathcal{O}(b)$ for a $b \times b$ flat tessellation.

When (2.2) holds, we obtain a block factorization of an $N \times N$ uniform BLR matrix $\mathbf{A}$ with b block-rows/columns of size $m \times m$ (so that $N = bm$), given by

$$(2.4) \quad \underset{bm \times bm}{\mathbf{A}} = \underset{bm \times bk}{\mathbf{U}} \underset{bk \times bk}{\tilde{\mathbf{A}}} \underset{bk \times bm}{\mathbf{V}}^* + \underset{bm \times bm}{\mathbf{B}},$$

where

$$(2.5) \quad \begin{aligned} \mathbf{U} &= \text{diag}(\mathbf{U}_1, \dots, \mathbf{U}_b), \\ \mathbf{V} &= \text{diag}(\mathbf{V}_1, \dots, \mathbf{V}_b), \end{aligned}$$

$$(2.6) \quad \tilde{\mathbf{A}} = \mathbf{U}^* \mathbf{A} \mathbf{V},$$

and $\mathbf{B}$ is a block-sparse matrix defined (for strongly admissible $\mathbf{A}$) as

$$(2.7) \quad \mathbf{B}_{i,j} = \begin{cases} \mathbf{A}_{i,j} - \mathbf{U}_i \tilde{\mathbf{A}}_{i,j} \mathbf{V}_j^*, & i \in \mathcal{N}_j \\ \mathbf{0}, & \text{otherwise.} \end{cases}$$

The matrix $\mathbf{B}$ given by (2.7) represents the “remainder” of the inadmissible blocks of $\mathbf{A}$ after their components spanned by the basis matrices have been peeled off.

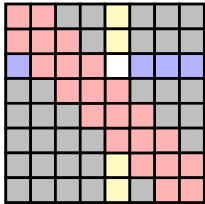
In general, uniform BLR compression can be accomplished by the following steps¹:

(I) Compute basis matrices $\mathbf{U}$ and $\mathbf{V}$.

(II) Compute matrix $\tilde{\mathbf{A}} = \mathbf{U}^* \mathbf{A} \mathbf{V}$.

(III) Compute discrepancy matrix $\mathbf{B} = \mathbf{A} - \mathbf{U} \tilde{\mathbf{A}} \mathbf{V}^*$.

The primary focus of this work is step (I): we develop and compare randomized algorithms for computing basis matrices of uniform BLR matrices under strong admissibility conditions. However, for completeness, we outline in Section 6 how the random sketches used for step (I) can be recycled for steps (II) and (III). First, we describe the two algorithms that we use to compute basis matrices: the method of block nullification in Section 3 and our improved method of tagging in Section 4.



For example, to compute $\mathbf{A}_{3,5} = \mathbf{U}_3 \tilde{\mathbf{A}}_{3,5} \mathbf{V}_5^*$ (white):

$$\begin{aligned} \mathbf{U}_3 &= \text{col} \left(\left[\begin{array}{c} \mathbf{A}_{3,1} \\ \mathbf{A}_{3,5} \\ \mathbf{A}_{3,6} \\ \mathbf{A}_{3,7} \\ \mathbf{A}_{3,8} \end{array} \right], k \right) \\ \mathbf{V}_5 &= \text{col} \left(\left[\begin{array}{c} \mathbf{A}_{1,5} \\ \mathbf{A}_{2,5} \\ \mathbf{A}_{3,5} \\ \mathbf{A}_{7,5} \\ \mathbf{A}_{8,5} \end{array} \right]^*, k \right) \\ \Rightarrow \tilde{\mathbf{A}}_{3,5} &= \mathbf{U}_3^* \mathbf{A}_{3,5} \mathbf{V}_5. \end{aligned}$$

Fig. 3: Basis matrices for a strongly-admissible uniform BLR matrix $\mathbf{A}$ (Fig. 2)

3. Block Nullification in Uniform BLR Matrix Compression. In this section, we modify the algorithm from [36, 58] using “block nullification” to form random sketches of admissible matrix blocks. These sketches are then used to compute basis matrices in an HBS representation according to the randomized rangefinding procedure of Section 2.4. The algorithm is also fully black-box, accomplishing steps (I)-(III) without access to matrix entries. Rather, it assumes access to fast matrix

¹Steps (II) and (III) are interchangeable depending on the compression algorithm; see Section 6.

multiplication, so that sample matrices $\mathbf{Y} = \mathbf{A}\mathbf{\Omega}$ and $\mathbf{Z} = \mathbf{A}^*\mathbf{\Psi}$ are formed efficiently given tall thin random test matrices $\mathbf{\Omega}, \mathbf{\Psi} \in \mathbb{R}^{N \times s}$ for $s = \mathcal{O}(k)$ for block-rank k .

The goal is then to “reconstruct” $\mathbf{A}$ via steps (I)-(III) using only $\mathbf{Y}, \mathbf{Z}, \mathbf{\Omega}$, and $\mathbf{\Psi}$ (computed a priori). However, each row of $\mathbf{Y}$, for instance, is a random linear combination of all columns of $\mathbf{A}$ within a block-row, including the columns belonging to inadmissible blocks. Block nullification yields “clean” random sketches from $\mathbf{Y}$ and $\mathbf{Z}$ by excluding contributions from inadmissible blocks, without repeatedly applying $\mathbf{A}$ or $\mathbf{A}^*$ to tailored random test matrices to individually sketch admissible blocks.

While block nullification is suitable for flat or hierarchical rank-structured formats, its performance has not been investigated for flat formats. As such, we first modify block nullification for strongly admissible uniform BLR matrices. We then discuss its asymptotic complexity to emphasize its larger pre-factor versus tagging.

3.1. Block nullification for strongly-admissible uniform BLR matrices.

We begin with an illustrative example of the block nullification technique applied to the strongly-admissible uniform BLR matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$ from Fig. 2, tessellated into $b \times b$ blocks each of size $m \times m$ and target rank k . Let $r = k + p$ for small p (e.g. $p = 10$), and let $\mathbf{\Omega}, \mathbf{\Psi} \in \mathbb{R}^{N \times s}$ be Gaussian test matrices with $s \geq r + 3m$.

Suppose that we want to compute $\mathbf{U}_3 \in \mathbb{R}^{m \times k}$ as in Fig. 3, now using the randomized rangefinder procedure with a random sketch $\mathbf{Y}$ of the form

$$\mathbf{Y} = \mathbf{A} \mathbf{\Omega}$$

where $\mathbf{Y}_3 = \mathbf{Y}(I_3, :)$, and $\mathbf{\Omega}_i = \mathbf{\Omega}(I_i, :)$ for $i = 1, \dots, 8$ are color-coded according to their respective block-factors in the block-row $\mathbf{A}_3 = \mathbf{A}(I_3, :)$. As in Fig. 3, the blue blocks are the admissible blocks in $\mathbf{A}_3$ whose columns will be approximately spanned by $\mathbf{U}_3$ computed with $\mathbf{Y}_3$, whereas the red blocks of $\mathbf{A}$ are inadmissible; our goal is to exclude their contributions from the randomized sample of $\mathbf{A}_3$ held in $\mathbf{Y}_3$.

Since $\mathbf{\Omega}^{(3)} := \mathbf{\Omega}([I_2, I_3, I_4], :)$ is of size $3m \times s$, it has a null space of dimension at least $s - 3m \geq r$. We then compute a set of r orthonormal vectors in its null space:

$$(3.1) \quad \mathbf{P}^{(3)}_{s \times r} = \text{null}(\mathbf{\Omega}^{(3)}, r) := \text{null}\left(\begin{bmatrix} \mathbf{\Omega}_2 \\ \mathbf{\Omega}_3 \\ \mathbf{\Omega}_4 \end{bmatrix}, r\right) \Rightarrow \mathbf{\Omega}^{(3)} \mathbf{P}^{(3)} = \begin{bmatrix} \mathbf{\Omega}_2 \\ \mathbf{\Omega}_3 \\ \mathbf{\Omega}_4 \end{bmatrix} \mathbf{P}^{(3)} = \mathbf{0}_{3m \times r}.$$

Thus, we can obtain the desired sample of the admissible blocks of $\mathbf{A}_3$ inexpensively from $\mathbf{Y}$ via $\mathbf{Y}_3 \mathbf{P}^{(3)}$ (blue):

$$\mathbf{Y} \mathbf{P}^{(3)} = \mathbf{A} \mathbf{\Omega} \mathbf{P}^{(3)}$$

noting that the white blocks of $\mathbf{\Omega} \mathbf{P}^{(3)}$ are filled with zeros. We also note that the blue blocks of $\mathbf{\Omega} \mathbf{P}^{(3)}$ contain standard Gaussian entries because (1) the distribution

of Gaussian matrices is invariant under unitary transformations and (2) the matrix $\mathbf{P}^{(3)}$ is computed independently of the blue blocks of $\mathbf{\Omega}$. Then we can compute $\mathbf{U}_3 = \text{col}(\mathbf{Y}_3 \mathbf{P}^{(3)}, k)$ with the usual probabilistic guarantees (cf. Section 2.4 and [27, 45]).

In general, block nullification computes basis matrices $\mathbf{U}, \mathbf{V}$ according to Algorithm 3.1 to accomplish step (I) of uniform BLR compression. Quickly summarizing, we first draw independent Gaussian matrices $\mathbf{\Omega}$ and $\mathbf{\Psi}$ to form random sketches $\mathbf{Y}$ and $\mathbf{Z}$ (lines 1-3). For any block $i = 1, \dots, b$, we define $\mathbf{\Omega}^{(i)}$ as the rows of $\mathbf{\Omega}$ indexed by $\{I_j\}_{j \in \mathcal{N}_i}$, or all I_j such that j is a neighbor of block i . We then compute r orthonormal vectors in the null space of $\mathbf{\Omega}^{(i)}$, which comprise the columns of $\mathbf{P}^{(i)}$. The matrix $\mathbf{Y}_i = \mathbf{Y}(I_i, :)$ is right-multiplied by $\mathbf{P}^{(i)}$ to compute $\mathbf{U}_i$, whose k columns form an approximate basis for the column space of $\mathbf{A}(I_i, :)$ excluding inadmissible blocks (lines 5-6). Analogously, for each block i , we compute the basis matrix $\mathbf{V}_i$ whose column space approximates the column space of $\mathbf{A}^*(I_i, :)$ excluding inadmissible blocks (lines 7-8). We discuss the asymptotic complexity of Algorithm 3.1 in the next section.

Algorithm 3.1 Block Nullification for Basis Construction

Require: Fast matrix-vector multiplication with uniform BLR $\mathbf{A} \in \mathbb{R}^{N \times N}$ and $\mathbf{A}^* \in \mathbb{R}^{N \times N}$, $b \times b$ tessellation, target rank k , max block-size m , d -dimensional geometry

Ensure: $\mathbf{U}, \mathbf{V} \in \mathbb{R}^{N \times bk}$ in uniform BLR representation of $\mathbf{A}$ as in (2.4)

- 1: Set $r = k + p$ and $s \geq r + 3^d m$
 - 2: Draw independent Gaussian matrices $\mathbf{\Omega}, \mathbf{\Psi} \in \mathbb{R}^{N \times s}$
 - 3: Sketch $\mathbf{Y} = \mathbf{A}\mathbf{\Omega}$ and $\mathbf{Z} = \mathbf{A}^*\mathbf{\Psi}$
 - 4: **for** blocks $i = 1, \dots, b$ **do**
 - 5: Compute $\mathbf{P}^{(i)} = \text{null}(\mathbf{\Omega}^{(i)}, r)$
 - 6: Compute $\mathbf{U}_i = \text{col}(\mathbf{Y}_i \mathbf{P}^{(i)}, k)$
 - 7: Compute $\mathbf{Q}^{(i)} = \text{null}(\mathbf{\Psi}^{(i)}, r)$
 - 8: Compute $\mathbf{V}_i = \text{col}(\mathbf{Z}_i \mathbf{Q}^{(i)}, k)$
 - 9: **end for**
 - 10: Set $\mathbf{U} = \text{diag}(\mathbf{U}_1, \dots, \mathbf{U}_b)$ and $\mathbf{V} = \text{diag}(\mathbf{V}_1, \dots, \mathbf{V}_b)$
-

3.2. Asymptotic complexity of block nullification. We now analyze the asymptotic complexity of Algorithm 3.1. Let $r = k + p$, where k is the block rank and p is the oversampling parameter, and let $s = 3^d m + r$, with m denoting the block size. For the purpose of generality, we do not assume that $\mathbf{A}$ is self-adjoint. However, if that is the case, there are savings of a factor of 2.

- **Gaussian matrix generation** (lines 2-3). To generate $\mathbf{\Omega}$ and $\mathbf{\Psi}$ we draw $2Ns$ values from the standard Gaussian distribution. The cost of this step is $2Ns \times T_{\text{rand}}$, where T_{rand} represents the time to draw a single value.
- **Matrix-vector products** (line 3). Forming $\mathbf{Y} = \mathbf{A}\mathbf{\Omega}$ and $\mathbf{Z} = \mathbf{A}^*\mathbf{\Psi}$ requires s matrix-vector multiplications for $\mathbf{A}$ and $\mathbf{A}^*$. This contributes $2s \times T_{\text{mult}}$ to the overall complexity, where T_{mult} is the cost of applying $\mathbf{A}$ or $\mathbf{A}^*$ to a vector.
- **Null-space bases** (lines 5, 7). For block $i = 1, \dots, b$, we compute $\mathbf{P}^{(i)} = \text{null}(\mathbf{\Omega}^{(i)}, r)$ and $\mathbf{Q}^{(i)} = \text{null}(\mathbf{\Psi}^{(i)}, r)$ with matrices of size at most $3^d m \times s$. Using Householder QR [29, Table C.2] for null , the cost of each is roughly $\mathcal{O}(3^{3d} m^3)$. For b blocks, this step adds $\mathcal{O}(b \times 3^{3d} m^3) \times T_{\text{flop}}$, where T_{flop} is the cost of one floating-point arithmetic operation.

- **Column basis extraction** (lines 6, 8). Computing $\mathbf{U}_i = \text{col}(\mathbf{Y}_i \mathbf{P}^{(i)}, k)$ and $\mathbf{V}_i = \text{col}(\mathbf{Z}_i \mathbf{Q}^{(i)}, k)$ involves matrix multiplications of dimensions $3^d m \times r$ by $r \times k$ for each block. The cost for all blocks is $\mathcal{O}(b \times [3^d m r^2]) \times T_{\text{flop}}$.
 - **Reconstruction.** Once $\mathbf{U}$ and $\mathbf{V}$ are computed, determining the matrix $\mathbf{B}$ for the full matrix reconstruction as in (2.4) involves additional matrix-vector products. The cost of this step is exactly $(3^d m + kb) \times T_{\text{mult}}$.
- Combining the contributions from all steps and using $N = mb$ gives total cost

$$2N(3^d m + r) \times T_{\text{rand}} + \mathcal{O}(N \times 3^{3d} m^2) \times T_{\text{flop}} + (3^{(d+1)} m + 2r + kN/m) \times T_{\text{mult}}.$$

4. Tagging in Uniform BLR Matrix Compression. We now introduce a black-box randomized method to compress strongly admissible uniform BLR matrices which we call tagging, the main contribution of our manuscript. As in Section 3, we begin with an illustrative example before generalizing the tagging method for d -dimensional problem geometries and summarizing its asymptotic complexity.

4.1. Tagging for strongly-admissible uniform BLR matrices. Let $\mathbf{A}$ be an $N \times N$ strongly admissible uniform BLR matrix as in Fig. 2, tessellated into $b \times b$ blocks each of size $m \times m$ with block rank k , allowing for a small amount of oversampling given by p , and let $r = k + p$. Our aim once again is to construct random test matrices $\mathbf{\Omega}, \mathbf{\Psi} \in \mathbb{R}^{N \times s}$ with $s = \mathcal{O}(k)$ so that the sketches $\mathbf{Y} = \mathbf{A}\mathbf{\Omega}$ and $\mathbf{Z} = \mathbf{A}^* \mathbf{\Psi}$ taken a priori can be used to compress the admissible blocks in every block-row and block-column.

To illustrate, suppose as in Section 3.1 that we want to compute the basis matrix $\mathbf{U}_3 \in \mathbb{R}^{m \times k}$ from Fig. 3. We first introduce the 8×4 *tagging matrix*

$$(4.1) \quad \mathbf{T} = \begin{bmatrix} t_{1,1} & t_{1,2} & t_{1,3} & t_{1,4} \\ t_{2,1} & t_{2,2} & t_{2,3} & t_{2,4} \\ \vdots & \vdots & \vdots & \vdots \\ t_{8,1} & t_{8,2} & t_{8,3} & t_{8,4} \end{bmatrix},$$

where the entries $t_{i,j}$ will be made explicit in Section 5; for now, we assume standard Gaussian entries. The number of rows of $\mathbf{T}$ equals the number of matrix blocks b . The number of columns of $\mathbf{T}$ is one more than the maximal number of neighbors within a given block-row/column, which is 4 for the strongly admissible 1-D matrix in Fig. 2. In general, for a d -dimensional problem geometry, the number of columns of $\mathbf{T}$ is $3^d + 1$.

We next define the *extended random test matrix* $\mathbf{\Omega} \in \mathbb{R}^{N \times 4r}$ in terms of random test matrices $\mathbf{\Omega}_j \in \mathbb{R}^{N \times r}$ for $j = 1, \dots, 4$, given by

$$(4.2) \quad \mathbf{\Omega}_{N \times 4r} := [\mathbf{\Omega}_1 \quad \mathbf{\Omega}_2 \quad \mathbf{\Omega}_3 \quad \mathbf{\Omega}_4] = \begin{bmatrix} t_{1,1} \mathbf{G}_1 & t_{1,2} \mathbf{G}_1 & t_{1,3} \mathbf{G}_1 & t_{1,4} \mathbf{G}_1 \\ t_{2,1} \mathbf{G}_2 & t_{2,2} \mathbf{G}_2 & t_{2,3} \mathbf{G}_2 & t_{2,4} \mathbf{G}_2 \\ t_{3,1} \mathbf{G}_3 & t_{3,2} \mathbf{G}_3 & t_{3,3} \mathbf{G}_3 & t_{3,4} \mathbf{G}_3 \\ t_{4,1} \mathbf{G}_4 & t_{4,2} \mathbf{G}_4 & t_{4,3} \mathbf{G}_4 & t_{4,4} \mathbf{G}_4 \\ \vdots & \vdots & \vdots & \vdots \\ t_{8,1} \mathbf{G}_8 & t_{8,2} \mathbf{G}_8 & t_{8,3} \mathbf{G}_8 & t_{8,4} \mathbf{G}_8 \end{bmatrix},$$

where each $\mathbf{G}_i \in \mathbb{R}^{m \times r}$, $i = 1, \dots, 8$, is a Gaussian random matrix, weighted by entry $t_{i,j}$ of the tagging matrix $\mathbf{T}$ to form $\mathbf{\Omega}_j$, $j = 1, \dots, 4$. Note that we again assume

303 $N = bm$ for notational convenience. We form the sketch matrix $\mathbf{Y} = \mathbf{A}\mathbf{\Omega} \in \mathbb{R}^{N \times 4r}$,
 304 partitioned into $N \times r$ block-columns commensurately with (4.2) so that

$$305 \quad \mathbf{Y}_{N \times 4r} = [\mathbf{Y}_1 \quad \mathbf{Y}_2 \quad \mathbf{Y}_3 \quad \mathbf{Y}_4] = [\mathbf{A}\mathbf{\Omega}_1 \quad \mathbf{A}\mathbf{\Omega}_2 \quad \mathbf{A}\mathbf{\Omega}_3 \quad \mathbf{A}\mathbf{\Omega}_4] = \mathbf{A}\mathbf{\Omega}.$$

306 To compute $\mathbf{U}_3 \in \mathbb{R}^{m \times k}$ as in Fig. 3, we exclude contributions from inadmissible
 307 blocks by computing a (nonzero) vector $\mathbf{z}^{(3)} = [z_1^{(3)} \quad z_2^{(3)} \quad z_3^{(3)} \quad z_4^{(3)}]^T$ so that

$$308 \quad (4.3) \quad \mathbf{z}^{(3)} = \text{null}(\mathbf{T}^{(3)}) := \text{null}\left(\begin{bmatrix} t_{2,1} & t_{2,2} & t_{2,3} & t_{2,4} \\ t_{3,1} & t_{3,2} & t_{3,3} & t_{3,4} \\ t_{4,1} & t_{4,2} & t_{4,3} & t_{4,4} \end{bmatrix}\right).$$

309 Note that this submatrix $\mathbf{T}^{(3)}$ of $\mathbf{T}$ comprises the rows that correspond to the neighbor
 310 list $\mathcal{N}_3 = \{2, 3, 4\}$ of box 3 (Fig. 2). Now consider the weighted sum

$$311 \quad z_1^{(3)}\mathbf{\Omega}_1 + z_2^{(3)}\mathbf{\Omega}_2 + z_3^{(3)}\mathbf{\Omega}_3 + z_4^{(3)}\mathbf{\Omega}_4 = \begin{bmatrix} (z_1^{(3)}t_{1,1} + z_2^{(3)}t_{1,2} + z_3^{(3)}t_{1,3} + z_4^{(3)}t_{1,4})\mathbf{G}_1 \\ (z_1^{(3)}t_{2,1} + z_2^{(3)}t_{2,2} + z_3^{(3)}t_{2,3} + z_4^{(3)}t_{2,4})\mathbf{G}_2 \\ (z_1^{(3)}t_{3,1} + z_2^{(3)}t_{3,2} + z_3^{(3)}t_{3,3} + z_4^{(3)}t_{3,4})\mathbf{G}_3 \\ (z_1^{(3)}t_{4,1} + z_2^{(3)}t_{4,2} + z_3^{(3)}t_{4,3} + z_4^{(3)}t_{4,4})\mathbf{G}_4 \\ (z_1^{(3)}t_{5,1} + z_2^{(3)}t_{5,2} + z_3^{(3)}t_{5,3} + z_4^{(3)}t_{5,4})\mathbf{G}_5 \\ \vdots \\ (z_1^{(3)}t_{8,1} + z_2^{(3)}t_{8,2} + z_3^{(3)}t_{8,3} + z_4^{(3)}t_{8,4})\mathbf{G}_8 \end{bmatrix}.$$

312 By construction, it simplifies to

$$(4.4) \quad 313 \quad z_1^{(3)}\mathbf{\Omega}_1 + z_2^{(3)}\mathbf{\Omega}_2 + z_3^{(3)}\mathbf{\Omega}_3 + z_4^{(3)}\mathbf{\Omega}_4 = \begin{bmatrix} (z_1^{(3)}t_{1,1} + z_2^{(3)}t_{1,2} + z_3^{(3)}t_{1,3} + z_4^{(3)}t_{1,4})\mathbf{G}_1 \\ \mathbf{0} \\ \mathbf{0} \\ \mathbf{0} \\ (z_1^{(3)}t_{5,1} + z_2^{(3)}t_{5,2} + z_3^{(3)}t_{5,3} + z_4^{(3)}t_{5,4})\mathbf{G}_5 \\ \vdots \\ (z_1^{(3)}t_{8,1} + z_2^{(3)}t_{8,2} + z_3^{(3)}t_{8,3} + z_4^{(3)}t_{8,4})\mathbf{G}_8 \end{bmatrix},$$

314 so that the rows of $\mathbf{A}(z_1^{(3)}\mathbf{\Omega}_1 + z_2^{(3)}\mathbf{\Omega}_2 + z_3^{(3)}\mathbf{\Omega}_3 + z_4^{(3)}\mathbf{\Omega}_4) = z_1^{(3)}\mathbf{Y}_1 + z_2^{(3)}\mathbf{Y}_2 + z_3^{(3)}\mathbf{Y}_3 +$
 315 $z_4^{(3)}\mathbf{Y}_4$ corresponding to I_3 will contain the desired sample of the admissible blocks,
 316 with the contributions from inadmissible blocks now zeroed out. We can then compute
 317 $\mathbf{U}_3 = \text{col}\left((z_1^{(3)}\mathbf{Y}_1 + z_2^{(3)}\mathbf{Y}_2 + z_3^{(3)}\mathbf{Y}_3 + z_4^{(3)}\mathbf{Y}_4)(I_3, :), k\right)$ as in Section 2.4.

318 In general, the tagging method computes basis matrices $\mathbf{U}$ and $\mathbf{V}$ according to
 319 Algorithm 4.1, which we quickly summarize. We begin by drawing entries of the
 320 tagging matrix $\mathbf{T} \in \mathbb{R}^{b \times (3^d+1)}$, e.g., from a standard Gaussian distribution (line 2).
 321 We next form the random sketches

$$(4.5) \quad 322 \quad \mathbf{Y}_{N \times (3^d+1)r} = [\mathbf{Y}_1 \quad \mathbf{Y}_2 \quad \dots \quad \mathbf{Y}_{3^d+1}] = [\mathbf{A}\mathbf{\Omega}_1 \quad \mathbf{A}\mathbf{\Omega}_2 \quad \dots \quad \mathbf{A}\mathbf{\Omega}_{3^d+1}] = \mathbf{A}\mathbf{\Omega},$$

$$(4.6) \quad 323 \quad \mathbf{Z}_{N \times (3^d+1)r} = [\mathbf{Z}_1 \quad \mathbf{Z}_2 \quad \dots \quad \mathbf{Z}_{3^d+1}] = [\mathbf{A}^*\mathbf{\Psi}_1 \quad \mathbf{A}^*\mathbf{\Psi}_2 \quad \dots \quad \mathbf{A}^*\mathbf{\Psi}_{3^d+1}] = \mathbf{A}^*\mathbf{\Psi}.$$

324 where each $\mathbf{\Omega}_j$, $j = 1, \dots, 3^d + 1$, comprises b block-rows of independent Gaussian
 325 matrices $\mathbf{G}_i \in \mathbb{R}^{m \times r}$, $i = 1, \dots, b$, weighted by tagging entry $t_{i,j}$ as in (4.2), and simi-
 326 larly for each $\mathbf{\Psi}_j$ with $\mathbf{H}_i$ (lines 3-8). To compute $\mathbf{U}_i$ or $\mathbf{V}_i$ for block i , we focus on the
 327 submatrix $\mathbf{T}^{(i)}$ comprising rows of $\mathbf{T}$ indexed by $\mathcal{N}_i$, the neighbor list of box i , so that
 328 $|\mathcal{N}_i| \leq 3^d$. We then compute an orthonormal tagging vector $\mathbf{z}^{(i)} = [z_1^{(i)} z_2^{(i)} \dots z_{3^d+1}^{(i)}]^T$
 329 in the nontrivial null space of $\mathbf{T}^{(i)}$ (line 10). Finally, we compute $\mathbf{U}_i, \mathbf{V}_i \in \mathbb{R}^{m \times k}$ (lines
 330 11-12) whose columns are an ON-basis approximating the column spaces of $\mathbf{A}(I_i, :)$
 331 and $\mathbf{A}^*(I_i, :)$, excluding contributions from the inadmissible blocks.

Algorithm 4.1 Tagging for Basis Construction

Require: Fast matrix-vector multiplication with uniform BLR $\mathbf{A} \in \mathbb{R}^{N \times N}$ and $\mathbf{A}^* \in \mathbb{R}^{N \times N}$, $b \times b$ tessellation, target rank k , max block-size m , d -dimensional geometry

Ensure: $\mathbf{U}, \mathbf{V} \in \mathbb{R}^{N \times bk}$ in uniform BLR representation of $\mathbf{A}$ as in (2.4)

- 1: Set $r = k + p$ and $\mathbf{\Omega}, \mathbf{\Psi} = [\]$.
 - 2: Form tagging matrix $\mathbf{T} \in \mathbb{R}^{b \times (3^d+1)}$ $\triangleright$ e.g. Gaussian $\mathbf{T}$; see Section 5
 - 3: **for** blocks $i = 1, \dots, b$ **do**
 - 4: Draw independent Gaussian matrices $\mathbf{G}_i, \mathbf{H}_i \in \mathbb{R}^{m \times r}$
 - 5: Update $\mathbf{\Omega} = [\mathbf{\Omega} \ ; \ [t_{i,1}\mathbf{G}_i \ \dots \ t_{i,3^d+1}\mathbf{G}_i]]$ $\triangleright$ Append i^{th} row to $\mathbf{\Omega}$
 - 6: Update $\mathbf{\Psi} = [\mathbf{\Psi} \ ; \ [t_{i,1}\mathbf{H}_i \ \dots \ t_{i,3^d+1}\mathbf{H}_i]]$ $\triangleright$ Append i^{th} row to $\mathbf{\Psi}$
 - 7: **end for**
 - 8: Sketch $\mathbf{Y} = \mathbf{A}\mathbf{\Omega}$ and $\mathbf{Z} = \mathbf{A}^*\mathbf{\Psi}$
 - 9: **for** blocks $i = 1, \dots, b$ **do**
 - 10: Compute $\mathbf{z}^{(i)} = \text{null}(\mathbf{T}^{(i)})$
 - 11: Compute $\mathbf{U}_i = \text{col}(\sum_{j=1}^{3^d+1} z_j^{(i)} \mathbf{Y}_j(I_i, :), k)$
 - 12: Compute $\mathbf{V}_i = \text{col}(\sum_{j=1}^{3^d+1} z_j^{(i)} \mathbf{Z}_j(I_i, :), k)$
 - 13: **end for**
 - 14: $\mathbf{U} = \text{diag}(\mathbf{U}_1, \dots, \mathbf{U}_b)$ and $\mathbf{V} = \text{diag}(\mathbf{V}_1, \dots, \mathbf{V}_b)$
-

332 **4.2. Asymptotic Complexity of Tagging.** We derive the asymptotic com-
 333 plexity of Algorithm 4.1 in terms of the problem size N , block size m , block-rank k ,
 334 oversampling parameter p , and problem geometry dimension d . Let $r = k + p$ and
 335 assume $N = mb$, where b is the number of matrix blocks in each block-row or block-
 336 column. The complexity of the algorithm is broken into the following components:

- 337 • **Tagging matrix formation** (line 2). The tagging matrix $\mathbf{T} \in \mathbb{R}^{b \times (3^d+1)}$
 338 requires $(3^d + 1)N/m$ elements to be sampled from a Gaussian distribution.
 339 The total complexity is $(3^d + 1)N/m \times T_{\text{rand}}$.
- 340 • **Random test matrix formation** (lines 3-7). Forming $\mathbf{\Omega}, \mathbf{\Psi} \in \mathbb{R}^{N \times (3^d+1)r}$
 341 first requires sampling $\mathbf{G}_i, \mathbf{H}_i$ for each subblock and weighting each by the
 342 relevant tag. This contributes $2Nr \times T_{\text{rand}}$, as each Gaussian matrix $\mathbf{G}_i$ and $\mathbf{H}_i$
 343 is reused. Forming $\mathbf{\Omega}, \mathbf{\Psi}$ then requires $2(3^d + 1)Nr$ floating-point operations.
 344 The total complexity is $2Nr \times T_{\text{rand}} + \mathcal{O}(3^d Nr) \times T_{\text{flop}}$.
- 345 • **Matrix-vector products** (line 8). Computing $\mathbf{Y} = \mathbf{A}\mathbf{\Omega}$ and $\mathbf{Z} = \mathbf{A}^*\mathbf{\Psi}$ in-
 346 volves $2(3^d + 1)r$ matrix-vector products, with complexity $2(3^d + 1)r \times T_{\text{mult}}$.
- 347 • **Null space bases** (line 10). For each block, the null space computation
 348 involves matrices of size at most $3^d \times (3^d + 1)$. Using Householder QR, the
 349 complexity is $\mathcal{O}(3^{3d}b) \times T_{\text{flop}}$.

- **Column basis extraction** (lines 11, 12). Forming $\mathbf{U}$ and $\mathbf{V}$ involves multiplying the relevant subblocks by the null space vector, then computing an ON-basis. This involves $(3^d+1)mr$ floating-point operations per block to scale submatrices of $\mathbf{Y}$ and $\mathbf{Z}$ and $\mathcal{O}(rmk)$ floating-point operations for $\mathbf{U}_i$ and $\mathbf{V}_i$. The total complexity is $\mathcal{O}(3^dNr + Nrk) \times T_{\text{flop}}$.
- **Reconstruction.** Reconstructing the uniform BLR matrix in (2.4) using $\mathbf{U}$ and $\mathbf{V}$ contributes an additional $(3^dm + kb) \times T_{\text{mult}}$.

Combining the above contributions, the total complexity is

$$\begin{aligned} & ((3^d + 1)N/m + 2Nr) \times T_{\text{rand}} + \mathcal{O}(3^dNr + Nrk + 3^{3d}N/m) \times T_{\text{flop}} \\ & + (2 \cdot 3^{d+1}r + 3^dm + kb) \times T_{\text{mult}}. \end{aligned}$$

Compared to block nullification in Section 3.2, far fewer samples are needed to construct $\mathbf{U}$ and $\mathbf{V}$. Block nullification requires 2×3^dm samples for basis construction in Section 3.2, whereas tagging only requires $2 \times 3^{d+1}r$ samples. The cost of reconstructing the uniform BLR matrix, however, dominates the asymptotic complexity of T_{mult} for both methods. The key advantage of tagging is the substantially reduced (linear) cost of post-processing the test and sketch matrices, as opposed to block-nullification, where post-processing scales linearly with N and quadratically with m .

5. Selecting the Tagging Matrix. In Section 4.1, we treated the tagging matrix as having standard Gaussian entries. However, standard Gaussian tagging matrix entries do not guarantee Gaussian samples of the input matrix. As such, we seek to address the following questions. *Does there exist an optimal tagging matrix? If not, can we still obtain a high-quality randomized embedding?*

To answer these questions, we first discuss our criteria for tagging matrix optimality in Section 5.1. We then present a conjecture on the existence of optimal tagging matrices in Section 5.2 which takes an algebraic-geometric perspective. We finish the section with a highly efficient alternative strategy to determine tagging matrices that perform well empirically despite their sub-optimality.

5.1. On the optimality of tagging matrices: Projected tags and aspect ratios. The main issue that we need to address in tagging matrix selection concerns the *projected tags*, the nonzero non-uniform weights on each Gaussian matrix in (4.4). These are given by $t_{j,1}z_1^{(i)} + t_{j,2}z_2^{(i)} + \dots + t_{j,\ell}z_\ell^{(i)}$ with $1 \leq j \leq b$ for each box $i = 1, \dots, b$. Here $\ell = 3^d + 1$ and $\mathbf{z}^{(i)} = [z_1^{(i)} \ z_2^{(i)} \ \dots \ z_\ell^{(i)}]^T = \text{null}(\mathbf{T}^{(i)})$ for $\mathbf{T}^{(i)} = \mathbf{T}(\mathcal{N}_i, :)$ where $\mathcal{N}_i$ is the neighbor list of box i . The projected tags for $\mathbf{G}_j$ are 0 by construction for any $j \in \mathcal{N}_i$. However, the nonzero projected tags, corresponding to the far-field $\mathcal{F}_i = [b] \setminus \mathcal{N}_i$ of box i , non-uniformly weight each Gaussian matrix, resulting in non-uniformly weighted randomized samples of blocks within the same block-row or block-column of $\mathbf{A}$. Each of the projected tags should ideally be equal in magnitude.

To this end, we define the *aspect ratio* $\rho^{(i)}$ for block-row/column $i = 1, \dots, b$ as the largest-magnitude to the smallest-magnitude nonzero projected tag:

$$\rho^{(i)} = \frac{\max_{j \in \mathcal{F}_i} |t_{j,1}z_1^{(i)} + t_{j,2}z_2^{(i)} + \dots + t_{j,\ell}z_\ell^{(i)}|}{\min_{j \in \mathcal{F}_i} |t_{j,1}z_1^{(i)} + t_{j,2}z_2^{(i)} + \dots + t_{j,\ell}z_\ell^{(i)}|} \geq 1.$$

An optimal tagging matrix then minimizes $\rho^{(i)}$ for each $i = 1, \dots, b$.

In the following subsections, we examine the tagging matrix optimality problem through two different lenses. The first relies on ideas from algebraic geometry to

determine an optimal tagging matrix. The second offers a numerical short-cut via null space vectors that minimize aspect ratios through a fast optimization scheme.

5.2. On the existence of optimal tagging matrices: Plücker coordinates.

To express our conjecture on optimal tagging matrices, we draw a connection between tagging matrices and projective varieties through *Plücker coordinates*. We return to our example of a uniform BLR matrix from Fig. 2 for an intuitive introduction to the Plücker embedding that gives rise to Plücker coordinates. We then hypothesize that optimal tagging matrices may be found through a hybrid numeric-symbolic approach based on Plücker coordinates, which is currently out of reach.

5.2.1. An illustrative example. Consider $\mathbf{A} \in \mathbb{R}^{N \times N}$ from Fig. 2. Note that for blocks $i = 2, \dots, 7^2$, we can apply Cramer's Rule to find $\mathbf{z}^{(i)} = \text{null}(\mathbf{T}^{(i)})$, e.g. the i^{th} coordinate $z_i^{(3)}$ of $\mathbf{z}^{(3)}$ is the determinant of $\mathbf{T}^{(3)}$ without the i^{th} column:

$$(5.2) \quad \begin{aligned} z_1^{(3)} &= \det \begin{pmatrix} t_{2,2} & t_{2,3} & t_{2,4} \\ t_{3,2} & t_{3,3} & t_{3,4} \\ t_{4,2} & t_{4,3} & t_{4,4} \end{pmatrix}, & z_2^{(3)} &= \det \begin{pmatrix} t_{2,1} & t_{2,3} & t_{2,4} \\ t_{3,1} & t_{3,3} & t_{3,4} \\ t_{4,1} & t_{4,3} & t_{4,4} \end{pmatrix}, \\ z_3^{(3)} &= \det \begin{pmatrix} t_{2,1} & t_{2,2} & t_{2,4} \\ t_{3,1} & t_{3,2} & t_{3,4} \\ t_{4,1} & t_{4,2} & t_{4,4} \end{pmatrix}, & z_4^{(3)} &= \det \begin{pmatrix} t_{2,1} & t_{2,2} & t_{2,3} \\ t_{3,1} & t_{3,2} & t_{3,3} \\ t_{4,1} & t_{4,2} & t_{4,3} \end{pmatrix}. \end{aligned}$$

Then the projected tag $\mathbf{Tz}^{(i)}$ contains all 4×4 determinants of $\mathbf{T}$ formed from the three rows of $\mathbf{T}^{(i)}$ plus one remaining row of $\mathbf{T}$, e.g. for $\mathbf{z}^{(3)}$ using (5.2),

$$(5.3) \quad \mathbf{Tz}^{(3)} = \begin{bmatrix} z_1^{(3)}t_{1,1} + z_2^{(3)}t_{1,2} + z_3^{(3)}t_{1,3} + z_4^{(3)}t_{1,4} \\ \vdots \\ z_1^{(3)}t_{8,1} + z_2^{(3)}t_{8,2} + z_3^{(3)}t_{8,3} + z_4^{(3)}t_{8,4} \end{bmatrix} = \begin{bmatrix} \det \begin{pmatrix} t_{1,1} & t_{1,2} & t_{1,3} & t_{1,4} \\ t_{2,1} & t_{2,2} & t_{2,3} & t_{2,4} \\ t_{3,1} & t_{3,2} & t_{3,3} & t_{3,4} \\ t_{4,1} & t_{4,2} & t_{4,3} & t_{4,4} \end{pmatrix} \\ \vdots \\ \det \begin{pmatrix} t_{2,1} & t_{2,2} & t_{2,3} & t_{2,4} \\ t_{3,1} & t_{3,2} & t_{3,3} & t_{3,4} \\ t_{4,1} & t_{4,2} & t_{4,3} & t_{4,4} \\ t_{8,1} & t_{8,2} & t_{8,3} & t_{8,4} \end{pmatrix} \end{bmatrix}.$$

When $\mathbf{T}$ has full rank, these 4×4 determinants form a subset of the *Plücker relations*, the set of all possible 4×4 determinants with the rows of $\mathbf{T}$. Let $\mathcal{L}$ be a 4-dimensional subspace of $\mathbb{R}^8$ with $\text{Col}(\mathbf{T}) = \mathcal{L}$. The *Plücker embedding* maps $\mathcal{L}$ to the point in real projective space whose coordinates are all 4×4 determinants of $\mathbf{T} \in \mathbb{R}^{8 \times 4}$. In other words, the Plücker embedding maps the *Grassmannian manifold* $\text{Gr}(4, 8)$, comprising all 4-dimensional subspaces of $\mathbb{R}^8$, to *Plücker coordinates* in $\mathbb{P}^{\binom{8}{4}-1}$, as stated below:

DEFINITION 5.1. The **Plücker embedding** is the map $\text{Gr}(k, n) \rightarrow \mathbb{P}^{\binom{n}{k}-1}$ that identifies $\mathcal{V} \in \text{Gr}(k, n)$ with a unique point in real projective space with coordinates given by all $k \times k$ determinants of a matrix $\mathbf{B} \in \mathbb{R}^{n \times k}$ satisfying $\mathcal{V} = \text{Col}(\mathbf{B})$, called **Plücker coordinates**.

²We treat the “extremal” blocks $i = 1$ and $i = 8$ (in general, blocks with fewer than the maximal number of neighbors 3^d) at the end of the section.

DEFINITION 5.2. Let $\mathcal{V} \in \text{Gr}(k, n)$ and suppose $\mathbf{B} \in \mathbb{R}^{n \times k}$ satisfies $\mathcal{V} = \text{Col}(\mathbf{B})$. For any ordered sequence of k row indices $1 \leq i_1 < \dots < i_k \leq n$ of $\mathbf{B}$, let $B_{i_1, \dots, i_k}$ be the determinant of $k \times k$ submatrix $\mathbf{B}([i_1, \dots, i_k], :)$, so that set of all Plücker coordinates may be denoted $\{B_{i_1, \dots, i_k}\}$. For any two ordered sequences of row indices

$$1 \leq i_1 < i_2 < \dots < i_{k-1} \leq n, \quad 1 \leq j_1 < j_2 < \dots < j_{k+1} \leq n,$$

the **Plücker relations** are the following homogeneous quadratic equations that must hold for all Plücker coordinates $\{B_{i_1, \dots, i_k}\}$:

$$(5.4) \quad \sum_{\ell=1}^{k+1} (-1)^\ell B_{i_1, \dots, i_{k-1}, j_\ell} B_{j_1, \dots, \widehat{j}_\ell, \dots, j_{k+1}} = 0,$$

where $j_1, \dots, \widehat{j}_\ell, \dots, j_{k+1}$ is the sequence $j_1, \dots, j_{k+1}$ with the term j_ℓ omitted.

We can now pose the optimality of tagging matrices in algebraic-geometric terms.

5.2.2. On the existence of optimal tagging matrices. Let $\mathbf{T} \in \mathbb{C}^{b \times 3^d + 1}$ be a matrix of indeterminates $t_{i,j}$. For any ordered sequence of $3^d + 1$ row indices $1 \leq i_1 < \dots < i_{3^d + 1} \leq b$ of $\mathbf{T}$, let $T_{i_1, \dots, i_{3^d + 1}}$ be the determinant of the $(3^d + 1) \times (3^d + 1)$ submatrix $\mathbf{T}([i_1, \dots, i_{3^d + 1}], :)$; e.g. $T_{2,3,4,8}$ is the 4×4 determinant in (5.3) involving $\mathbf{T}^{(3)}$ and the last row of $\mathbf{T}$. Each $T_{i_1, \dots, i_{3^d + 1}}$ is a degree- $(3^d + 1)$ polynomial in the indeterminates $t_{i,j}$, and from the previous section, each is a Plücker coordinate that must satisfy the Plücker relations, which are quadratic polynomials in the indeterminates $T_{i_1, \dots, i_{3^d + 1}}$ for all possible ordered sequences $1 \leq i_1 < \dots < i_{3^d + 1} \leq b$.

To determine tagging matrix entries $t_{i,j}$ that minimize $\rho^{(i)}$ for each block i , we propose the following approach. It is well-known that the set of Plücker relations is not algebraically independent, cf. [47, Chapter 14.2] and [28, Appendix C.7]. Thus, the first step is to determine an algebraically independent generating set of Plücker relations for $\mathbf{T}$. One method is the computation of a Gröbner basis³ for the ideal of the polynomial ring $\mathbb{C}[\{t_{i,j}\}_{i \in [b], j \in [3^d + 1]}]$ generated by all Plücker relations, cf. [49].

Remark 5.3. Another avenue of investigation that bears future consideration involves the so-called “clusters” formed by independent Plücker coordinates [51]. One such cluster is comprised of *rectangular Plücker coordinates* [35], which correspond to the rectangular partitions of a $k \times (n - k)$ unit rectangle and form a generating set for the coordinate ring. Rectangular Plücker coordinates relate to quantum Schubert calculus on the flag variety [6, 13, 19] and have an associated Laurent polynomial with certain properties [39] that may offer another path to an optimal tagging matrix.

Let $\mathcal{R}$ denote a set of algebraically independent Plücker relations that generate the projective variety defined by them all. For the purposes of tagging, we are only interested in the nonzero projected tags, which correspond to the far-field of each box. Let n be the total number of nonzero projected tags, with n_i nonzero projected tags for block i , so that $\sum_{i=1}^b n_i = n$. Denote the n nonzero projected tags by $\{T_1, \dots, T_n\}$:

$$(5.5) \quad \{T_1, \dots, T_n\} := \left\{ T_{i_1^{(1)}, \dots, i_{3^d+1}^{(1)}}, T_{i_1^{(2)}, \dots, i_{3^d+1}^{(2)}}, \dots, T_{i_1^{(n)}, \dots, i_{3^d+1}^{(n)}} \right\}$$

for distinct ordered sequences $1 \leq i_1^{(\ell)} < \dots < i_{3^d+1}^{(\ell)} \leq b$ for $\ell = 1, \dots, n$.

³The computation of a Gröbner basis for the quadratic polynomials under consideration is highly nontrivial for problems of this size due to, e.g., intermediate swell in Buchberger’s algorithm; see [12].

457 Ideally, for each block i , every nonzero projected tag, or nonzero coordinate of
 458 $\mathbf{T}\mathbf{z}^{(i)}$, should be equal (cf. (5.1)), which we can enforce numerically via

$$459 \quad (5.6) \quad \min_{\mathbf{T} \in \mathbb{R}^{b \times 3^d + 1}} \sum_{i=1}^b \sum_{\ell_i=1}^{n_i} (T_{\ell_i} - \bar{T}_i)^2$$

subject to $\mathcal{R}$

460 where $\bar{T}_i$ denotes the mean of the n_i nonzero projected tags for block i .

461 We now note that this approach only holds for blocks with neighbor lists of
 462 maximal size 3^d . One workaround is to treat the necessary number of admissible
 463 blocks as inadmissible in the block-rows/columns of $\mathbf{A}$ with fewer than 3^d inadmissible
 464 blocks, though this increases the overall cost. More detrimental, though, is the cost
 465 of the symbolic computations. The computations for $\mathcal{R}$ are highly nontrivial even for
 466 very small values of b , constraints which must hold if the $\arg \min \mathbf{T}^*$ of (5.6) satisfies
 467 $\text{Col}(\mathbf{T}^*) = \mathcal{V}$ for some $(3^d + 1)$ -dimensional subspace $\mathcal{V}$ of $\mathbb{R}^b$. Moreover, if the problem
 468 size or underlying geometry were to change, these computations would need to be done
 469 anew for different values of b or d . We next describe a strictly numerical method to
 470 minimize the aspect ratios that performs very well in practice.

471 **5.3. A practical method for numerical optimization.** Because of the prac-
 472 tical difficulties in using the approach of Section 5.2, we present an alternative to min-
 473 imize the aspect ratios numerically. This approach requires higher-dimensional null
 474 spaces of tagging submatrices; as such, we now consider tagging matrices $\mathbf{T} \in \mathbb{R}^{b \times \ell}$
 475 where $\ell > 3^d + 1$ so that every null space has dimension strictly greater than 1.

476 To minimize the aspect ratios of projected tags efficiently, we consider the follow-
 477 ing optimization problem over the $(\ell - 3^d)$ -dimensional null space of $\mathbf{T}^{(i)} \in \mathbb{R}^{3^d \times \ell}$,
 478 rather than over all possible matrix representations $\mathbf{T}$ of a Grassmannian subspace
 479 $\mathcal{V}$ such that $\mathcal{V} = \text{Col}(\mathbf{T})$. We now seek a unit vector $\mathbf{z}^{(i)}$ in the null space of $\mathbf{T}^{(i)}$ for
 480 each block $i = 1, \dots, b$, which minimizes the ratio of projected tags:

$$481 \quad (5.7) \quad \mathbf{z}^{(i)} = \arg \min_{\mathbf{x} \in \text{Null}(\mathbf{T}^{(i)})} \frac{\max_{j \in \mathcal{F}_i} |t_{j,1}x_1^{(i)} + t_{j,2}x_2^{(i)} + \dots + t_{j,\ell}x_\ell^{(i)}|}{\min_{j \in \mathcal{F}_i} |t_{j,1}x_1^{(i)} + t_{j,2}x_2^{(i)} + \dots + t_{j,\ell}x_\ell^{(i)}|}.$$

482 For arbitrary $i = 1, \dots, b$ with $\mathbf{T} \in \mathbb{R}^{b \times \ell}$ and $\ell > 3^d + 1$, the null space of
 483 submatrix $\mathbf{T}^{(i)}$ has dimension at least $s = \ell - 3^d > 1$. We compute

$$484 \quad \begin{bmatrix} \mathbf{x}_1^{(i)} & \dots & \mathbf{x}_s^{(i)} \end{bmatrix} = \text{null} \left(\mathbf{T}^{(i)}, s \right),$$

485 and write any normalized vector in the null space as

$$486 \quad \mathbf{x} = \alpha_1(\boldsymbol{\theta})\mathbf{x}_1^{(i)} + \dots + \alpha_s(\boldsymbol{\theta})\mathbf{x}_s^{(i)}$$

487 for (spherical) coordinates $(\alpha_1(\boldsymbol{\theta}), \dots, \alpha_s(\boldsymbol{\theta})) \in \mathbb{R}^s$ parameterizing the unit hyper-
 488 sphere over $\boldsymbol{\theta} \in \mathbb{R}^{s-1}$. We then solve the small constrained optimization problem

$$489 \quad \boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta} \in \mathbb{R}^{s-1}} \frac{\max_{j \in \mathcal{F}_i} \left| \left(\alpha_1(\boldsymbol{\theta})\mathbf{x}_1^{(i)} + \dots + \alpha_s(\boldsymbol{\theta})\mathbf{x}_s^{(i)} \right)^T \mathbf{t}^{(j)} \right|}{\min_{j \in \mathcal{F}_i} \left| \left(\alpha_1(\boldsymbol{\theta})\mathbf{x}_1^{(i)} + \dots + \alpha_s(\boldsymbol{\theta})\mathbf{x}_s^{(i)} \right)^T \mathbf{t}^{(j)} \right|},$$

where $\mathbf{t}^{(j)} = [t_{j,1} \ t_{j,2} \ \dots \ t_{j,\ell}]^T$, to write our desired null space vector as

$$\mathbf{z}^{(i)} = \alpha_1(\boldsymbol{\theta}^*)\mathbf{x}_1^{(i)} + \dots + \alpha_s(\boldsymbol{\theta}^*)\mathbf{x}_s^{(i)},$$

and compute the basis matrices $\mathbf{U}_i, \mathbf{V}_i$ as in Section 4.

The overall computational cost of this optimization procedure is negligible when it is integrated into Algorithm 4.1, since the optimization happens over a convex region, with no more than a 2-dimensional parameterization in practice. However, each additional column in the tagging matrix corresponds to $k + p$ additional samples of $\mathbf{A}$ and $\mathbf{A}^*$ with $\boldsymbol{\Omega}$ and $\boldsymbol{\Psi}$, though the total is still far fewer than in block nullification which we verify numerically in Section 7. First, for completeness, we outline our method of reconstructing the full uniform BLR representation of (2.4).

6. Randomized Compression Algorithms for Uniform BLR Matrices.

In the previous sections, we focused on step (I) of randomized compression—the computation of basis matrices $\mathbf{U}, \mathbf{V}$ —due to the similarity of compression algorithms after step (I). More precisely, steps (II) and (III) of uniform BLR compression can be executed in a manner that is oblivious to the particular method used for step (I), allowing for direct performance comparisons of Algorithms 3.1 and 4.1.

In this section, we briefly describe how steps (I)-(III) are conducted in our experiments. Remark 6.1 also summarizes the situation when matrix entries are readily available. First, Algorithm 6.1 introduces the last basis construction algorithm we use as a benchmark for step (I), which is equivalent to a blocked version of the randomized SVD done “naively” with $O(bk)$ structured Gaussian test matrices. We then describe a uniform BLR compression procedure that can be performed with basis matrices obtained from any of our algorithms; we discuss compression algorithms that reuse the sketches from step (I) for steps (II) and (III) in the Supplementary Materials.

Remark 6.1. When matrix entries are available, the task of recovering a uniform BLR representation as in (2.4) can be done with $k + p$ matvecs of $\mathbf{A} - \mathbf{D}$ and $\mathbf{A}^* - \mathbf{D}^*$, where $\mathbf{D}$ is given by $\mathbf{D}_{i,j} = \mathbf{A}_{i,j}$, for $i = 1, \dots, b$ and $j \in \mathcal{N}_i$. The recovery of $\tilde{\mathbf{A}}$ is then done by accessing $O(bk)$ matrix entries if $\mathbf{U}$ and $\mathbf{V}$ are computed as interpolative bases; see [44, Chapter 18] for more details.

Algorithm 6.1 Naive RandSVD for Basis Construction

Require: Fast matrix-vector multiplication with uniform BLR $\mathbf{A} \in \mathbb{R}^{N \times N}$ and $\mathbf{A}^* \in \mathbb{R}^{N \times N}$, $b \times b$ tessellation, target rank k , max block-size m , d -dimensional geometry

Ensure: $\mathbf{U}, \mathbf{V} \in \mathbb{R}^{N \times bk}$ in uniform BLR representation of $\mathbf{A}$ as in (2.4)

- 1: Set $r = k + p$
 - 2: **for** blocks $i = 1, \dots, b$ **do**
 - 3: Draw independent Gaussian test matrices $\boldsymbol{\Omega}, \boldsymbol{\Psi} \in \mathbb{R}^{N \times r}$
 - 4: Set $\boldsymbol{\Omega}(I_j, :) = 0$ and $\boldsymbol{\Psi}(I_j, :) = 0$ for all $j \in \mathcal{N}_i$
 - 5: Form $\mathbf{Y} = \mathbf{A}\boldsymbol{\Omega}$ and $\mathbf{Z} = \mathbf{A}^*\boldsymbol{\Psi}$
 - 6: Compute $\mathbf{U}_i = \text{col}(\mathbf{Y}(I_i, :), k)$
 - 7: Compute $\mathbf{V}_i = \text{col}(\mathbf{Z}(I_i, :), k)$
 - 8: **end for**
 - 9: Set $\mathbf{U} = \text{diag}(\mathbf{U}_1, \dots, \mathbf{U}_b)$ and $\mathbf{V} = \text{diag}(\mathbf{V}_1, \dots, \mathbf{V}_b)$
-

6.1. Direct evaluation in steps (II) and (III). We present algorithms to compress uniform BLR matrices that allow for the most direct comparison of our

basis construction algorithms. First, we summarize in Algorithm 6.1 an algorithm used only as a benchmark for step (I), which is equivalent to a blocked version of the randomized SVD done “naively” with $O(bk)$ structured Gaussian test matrices.

Steps (II) and (III) for (2.4) can be performed straightforwardly by evaluating $\tilde{\mathbf{A}} = \mathbf{U}^*(\mathbf{A}\mathbf{V})$ directly, for bk additional matvecs with $\mathbf{A}$. Then 3^d sparse structured matrices of size $N \times m$, containing the $m \times m$ identity as submatrices, can be used to extract the nonzero entries of $\mathbf{A} - \mathbf{U}\tilde{\mathbf{A}}\mathbf{V}^*$ for $\mathbf{B}$ as in [38], e.g.

$$(6.1) \quad \mathbf{Y}_B = (\mathbf{A} - \mathbf{U}\tilde{\mathbf{A}}\mathbf{V}^*) \Omega_B$$

Table 1 summarizes each reconstruction algorithm, which we call type A, vs. type B and C in the Supplementary Materials. Steps 2–3 in Table 1 are all identical, making type A ideal for performance comparisons of our basis computation algorithms.

<u>Algorithm A1</u>	<u>Algorithm A2</u>	<u>Algorithm A3</u>
1. Execute Alg. 3.1	1. Execute Alg. 4.1	1. Execute Alg. 6.1
2. Form $\tilde{\mathbf{A}} = \mathbf{U}^*(\mathbf{A}\mathbf{V})$	2. Form $\tilde{\mathbf{A}} = \mathbf{U}^*(\mathbf{A}\mathbf{V})$	2. Form $\tilde{\mathbf{A}} = \mathbf{U}^*(\mathbf{A}\mathbf{V})$
3. Form $\mathbf{B}$ as in (6.1)	3. Form $\mathbf{B}$ as in (6.1)	3. Form $\mathbf{B}$ as in (6.1)

Table 1: Randomized compression algorithms of type A for uniform BLR format of (2.4). Steps 2–3 contribute an additional bk and $3^d m$ matvecs with $\mathbf{A}$, respectively.

7. Numerical Experiments. We now demonstrate the improved performance of tagging over block nullification to compress strongly admissible uniform BLR matrices. For several different test problems and problem sizes N , we report the following:

- Accuracy of compressed $\mathbf{A}_{\text{uBLR}}$ of the form (2.4), using the relative error metric $\frac{\|\mathbf{A} - \mathbf{A}_{\text{uBLR}}\|_2}{\|\mathbf{A}\|_2}$ via 20 iterations of the randomized power method [27],
- Total runtime (in seconds) of each compression algorithm,
- Total number of matvecs with $\mathbf{A}$, $\mathbf{A}^*$ required for compression.

For tagging, we also report the aspect ratios after performing the optimization routine of Section 5.3. All experiments were performed with Gaussian tagging matrices with 3 additional columns; alternative tagging matrices are discussed in the Supplementary Materials. All test problems were implemented in MATLAB 2024a, and all experiments were carried out on a workstation with an Intel(R) Xeon(R) Gold 6254 CPU operating at 3.10GHz with 72 cores and 750 GB of memory.

For each test problem, we use a target block-rank of $k = 30$ with an oversampling parameter of $p = 10$. We also report the number of blocks b and the maximum block size m for each problem size N . To substantiate our choices of b and m , we highlight a

key distinction between hierarchical and flat rank-structured matrix formats. Often in hierarchical rank-structured matrix compression, the leaf node size m is chosen such that $m = O(k+p)$ with approximately $2N/m$ total nodes in the index tree; moreover, linear complexity can be achieved by leveraging nested bases, e.g. [36].

By contrast, randomized compression of flat formats does not attain linear complexity. The storage requirement of a strongly admissible uniform BLR matrix is $M \sim Nk + b^2 k^2 + 3^d N^2/b$ floating point values, and the compressed uniform BLR representation $\mathbf{A}_{\text{uBLR}}$ of (2.4) can be recovered in no fewer than $N_{\text{matvec}} \sim M/N$ matvecs, since a matrix of size $N \times N_{\text{matvec}}$ holds the *minimum* number of entries needed to store $\mathbf{A}_{\text{uBLR}}$. The dominant storage costs can be attributed to $\mathbf{A}$ and $\mathbf{B}$, and it is challenging to recover them in an “optimal” number of matvecs ($N_{\text{matvec}} \sim 3^d m + bk$). Thus, we choose b to balance N_{matvec} so that $b = \sqrt{3^d N/k} \Rightarrow M \sim \sqrt{3^d k} N^{3/2}$, a reasonable choice for medium-sized problems, e.g. $20,000 \leq N \leq 100,000$ here.

7.1. 2D Laplace Kernel. To profile performance, we use the Green’s function of the 2D Laplace equation for uniformly random $\{x_i\}_{i=1}^N$ in the unit square, where

$$(7.1) \quad \mathbf{A}_{ij} = \log(\|x_i - x_j\|), \text{ for } i \neq j,$$

and entries on the diagonal are set to 0. Dense systems of this form commonly arise in the context of integral equations. In practice, the method of proxy surfaces is often used to approximate basis matrices algebraically [11, 55], but we include this problem as a benchmark because the algebraic rank behavior of $\mathbf{A}$ is well-characterized by multipole estimates [22, 23] and exhibits exponential decay.

Fig. 4 summarizes the results. Matrix-vector products $\mathbf{x} \rightarrow \mathbf{A}\mathbf{x}$ are performed using FMM2D, a Fortran implementation of the fast multipole method developed and maintained by the Flatiron Institute. Key observations are as follows:

- Tagging significantly reduces the number of matrix-vector products (N_{matvec}) required for basis construction compared to alternative methods. For Algorithms A1 and A3, N_{matvec} scales with the block size and the number of blocks, respectively—both of which grow with the problem size N in flat formats. In contrast, the number of matvecs for tagging remains constant.
- The reduction in N_{matvec} for basis construction translates into considerable time savings during the sketching of $\mathbf{Y}$ and $\mathbf{Z}$.
- Tagging saves substantially on post-processing time vs. block nullification.
- Algorithm A2 (with tagging) achieves an 8.3x reduction in the total number of matvecs for the largest problem size compared to naive matrix formation.
- Tagging maintains comparable accuracy to alternative methods, and the aspect ratios can be effectively controlled by the method of Section 5.3.

7.2. Sparse Schur Complement for a Thin Slab. In sparse direct solvers for elliptic PDEs, compressing and factorizing sparse matrices is often necessary. Accessing matrix entries directly is computationally challenging, and randomized sketching techniques are frequently used to accelerate and simplify nested dissection solvers.

Consider solving the constant-coefficient Helmholtz equation with zero body load and prescribed Dirichlet boundary conditions on a domain Ω :

$$(7.2) \quad \begin{aligned} -\Delta u(x) - \kappa^2 u(x) &= 0, & x \in \Omega, \\ u(x) &= g(x), & x \in \partial\Omega. \end{aligned}$$

Discretizing with second-order finite differences leads to the linear system $\mathbf{A}\mathbf{u} = \mathbf{f}$ to solve. To solve this system efficiently, the domain Ω is partitioned into *thin slabs*, where one dimension is constrained to be electrically small, as demonstrated in [59, 14].

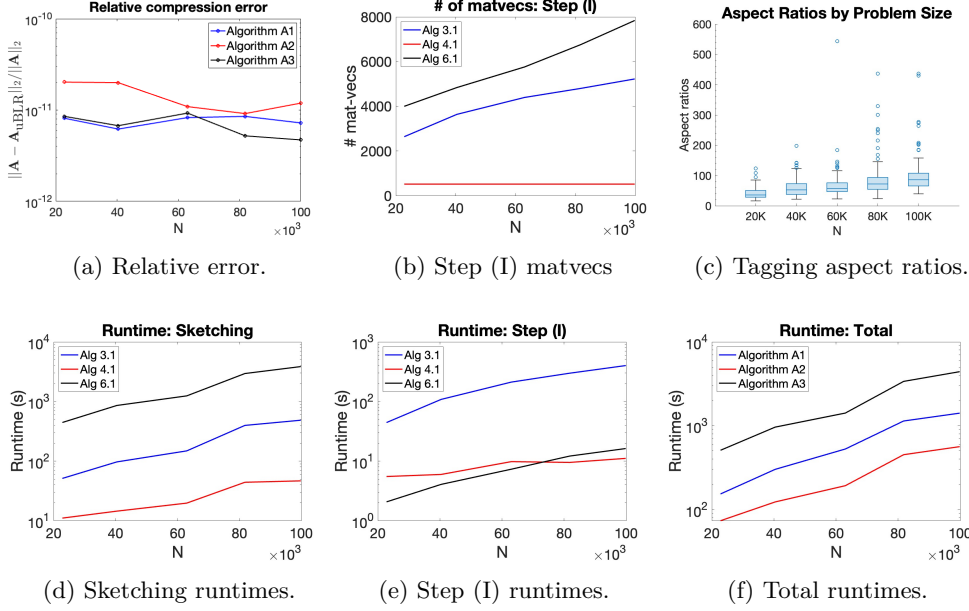


Fig. 4: Accuracy, number of matvecs, and timing results for 2D Laplace FMM in Section 7.1. Fig. 4a reports the reconstruction accuracy for algorithmic variants. Fig. 4b reports the number of matrix-vector products for step (I). Fig. 4c shows box-plots of the tagging aspect ratios. Figs. 4d-4f report algorithmic runtimes.

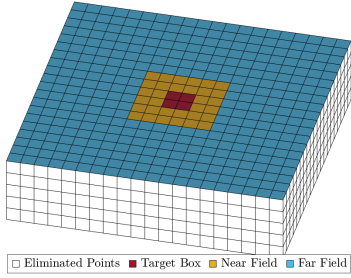


Fig. 5: The discretization of a thin slab of size $n \times n \times l$, where $l \ll n$ is fixed to be $l = 10$ in our experiments. The eliminated discretization points are shown in white, and the frontal indices are colored to denote the far field and near field for a target box.

We investigate the use of uniform BLR matrices for a slab subdomain. For a domain with $n \times n \times l$ discretization points, where l is fixed to be $l = 10$, the front size grows as $N = n^2$ as visualized in Fig. 5. The wavenumber parameter scales with the number of discretization points to maintain 100 points per wavelength. For the largest problem size ($N = 100,000$), the domain measures approximately $3\lambda \times 3\lambda \times 0.1\lambda$, where λ denotes the wavelength. In the context of domain decomposition, the degrees of freedom are partitioned into frontal nodes and internal nodes. The Schur complement is the linear algebraic operator that eliminates the internal nodes in a multifrontal solver, resulting in a dense matrix defined on the frontal nodes. While the Schur complement is dense, it can be applied efficiently to vectors by leveraging the sparsity of its components [59], enabling fast application of the solver to vectors.

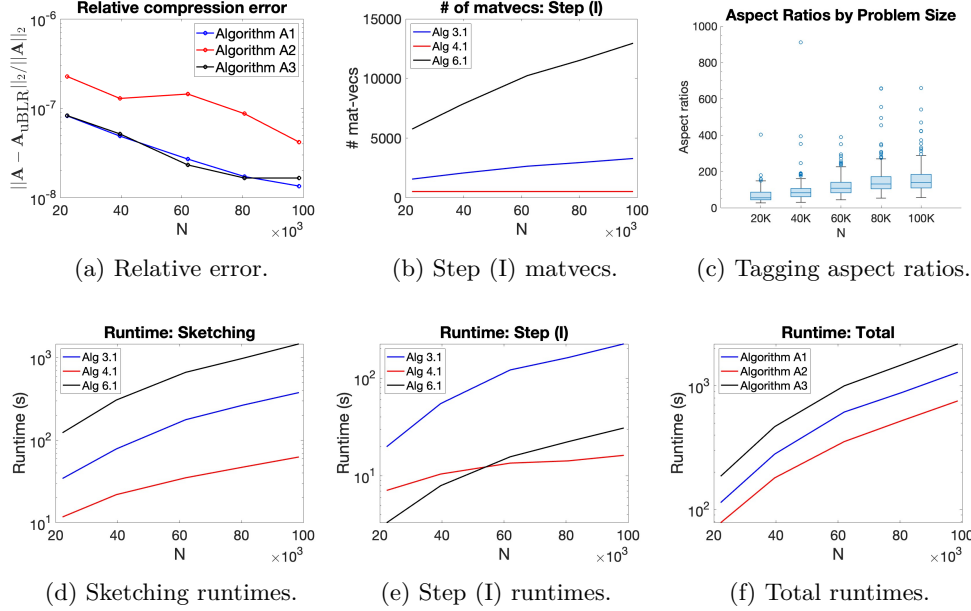


Fig. 6: Accuracy, number of matvecs, and timing results for the Schur complement in Section 7.2. Fig. 6a reports the reconstruction accuracy for algorithmic variants. Fig. 6b reports the number of matrix-vector products for step (I). Fig. 6c shows boxplots of tagging aspect ratios. Figs. 6d-6f report algorithmic runtimes.

Fig. 6 shows that tagging provides excellent scaling in the number of samples needed for step (I). It is also the most computationally efficient. As in Section 7.1, we observe that the far-field rank decays exponentially. Since the slab width is fixed, the rank of far-field interactions decays *faster* as the problem size grows, leading to improved approximation accuracy for increasing N and a fixed rank k . As in Section 7.1, the tagging aspect ratios can be effectively controlled by the method of Section 5.3.

8. Conclusions and Future Work. We present a black-box randomized compression algorithm based on tagging, improving on existing randomized compression algorithms for uniform BLR matrices under a strong admissibility condition. Our method only requires $O(k)$ random samples for basis computations, versus $O(m + k)$ for block nullification (which increases with problem size N for flat formats). We show that compression with tagging achieves comparable accuracy to existing compression algorithms with greatly improved computational efficiency. We also draw a connection between optimality in tagging and Plücker coordinates, and we present an alternative fast method of optimizing tagging matrices that is reliable in practice.

Avenues of future work include a hybrid numeric-symbolic computational scheme to generate theoretically optimal tagging matrix entries. Additionally, we are developing a high-performance implementation of our randomized compression algorithm with tagging. Future work will also investigate tagging for hierarchical formats.

Acknowledgements. The authors wish to thank Joe Kileel and Steven Karp for their invaluable insights.

REFERENCES

- [1] K. AKBUDAK, H. LTAIEF, A. MIKHALEV, AND D. KEYES, Tile low rank cholesky factorization for climate/weather modeling applications on manycore architectures, in High Performance Computing, J. M. Kunkel, R. Yokota, P. Balaji, and D. Keyes, eds., Cham, 2017, Springer International Publishing, pp. 22–40.
- [2] P. AMESTOY, C. ASHCRAFT, O. BOITEAU, A. BUTTARI, J.-Y. L'EXCELLENT, AND C. WEISBECKER, Improving Multifrontal Methods by Means of Block Low-Rank Representations, SIAM Journal on Scientific Computing, 37 (2015), pp. A1451–A1474.
- [3] P. AMESTOY, A. BUTTARI, J.-Y. L'EXCELLENT, AND T. A. MARY, On the Complexity of the Block Low-Rank Multifrontal Factorization, SIAM Journal on Scientific Computing, 39 (2017), pp. A1710–A1740.
- [4] P. R. AMESTOY, A. BUTTARI, J.-Y. L'EXCELLENT, AND T. A. MARY, Bridging the gap between flat and hierarchical low-rank matrix formats: The multilevel block low-rank format, SIAM Journal on Scientific Computing, 41 (2019), pp. A1414–A1442.
- [5] C. ASHCRAFT, A. BUTTARI, AND T. MARY, Block low-rank matrices with shared bases: Potential and limitations of the BLR² format, SIAM Journal on Matrix Analysis and Applications, 42 (2021), pp. 990–1010.
- [6] V. V. BATYREV, I. CIOCAN-FONTANINE, B. KIM, AND D. VAN STRATEN, Mirror symmetry and toric degenerations of partial flag manifolds, Acta Mathematica, 184 (2000), pp. 1–39.
- [7] M. BEBENDORF, Hierarchical matrices, Springer, 2008.
- [8] S. BÖRM, Efficient numerical methods for non-local operators, vol. 14 of EMS Tracts in Mathematics, European Mathematical Society (EMS), Zürich, 2010. $\mathcal{H}^2$ -matrix compression, algorithms and analysis.
- [9] Q. CAO, Y. PEI, K. AKBUDAK, A. MIKHALEV, G. BOSILCA, H. LTAIEF, D. KEYES, AND J. DONGARRA, Extreme-scale task-based Cholesky factorization toward climate and weather prediction applications, in Proceedings of the Platform for Advanced Scientific Computing Conference, PASC '20, New York, NY, USA, 2020, Association for Computing Machinery.
- [10] S. CHANDRASEKARAN, M. GU, AND T. PALS, A fast ULV decomposition solver for hierarchically semiseparable representations, SIAM Journal on Matrix Analysis and Applications, 28 (2006), pp. 603–622.
- [11] H. CHENG, Z. GIMBUTAS, P.-G. MARTINSSON, AND V. ROKHLIN, On the compression of low rank matrices, SIAM Journal on Scientific Computing, 26 (2005), pp. 1389–1404.
- [12] A. DEMIN AND S. GOWDA, Groebner.jl: A package for Gröbner bases computations in Julia, arXiv preprint, abs/2304.06935 (2023).
- [13] T. EGUCHI, K. HORI, AND C. SHENG XIONG, Gravitational quantum cohomology, Int. J. Mod. Phys., A12 (1997), pp. 1743–1782.
- [14] B. ENGQUIST AND L. YING, Sweeping preconditioner for the helmholtz equation: hierarchical matrix representation, Communications on pure and applied mathematics, 64 (2011), pp. 697–735.
- [15] A. GEORGE, Nested dissection of a regular finite element mesh, SIAM Journal on Numerical Analysis, 10 (1973), pp. 345–363.
- [16] K. GIEBERMANN, Multilevel approximation of boundary integral operators, Computing, 67 (2001), pp. 183–207.
- [17] J. R. GILBERT AND R. E. TARJAN, The analysis of a nested dissection algorithm, Numerische Mathematik, 50 (1986), pp. 377–404.
- [18] A. GILLMAN, P. M. YOUNG, AND P.-G. MARTINSSON, A direct solver with $\mathcal{O}(n)$ complexity for integral equations on one-dimensional domains, Frontiers of Mathematics in China, 7 (2012), pp. 217–247.
- [19] A. B. GIVENTAL, Stationary phase integrals, quantum toda lattices, flag manifolds and the mirror conjecture, Topics in singularity theory, American Mathematical Society Translations Ser 2., 180 (1997).
- [20] G. H. GOLUB AND C. F. VAN LOAN, Matrix computations, JHU press, 2013.
- [21] C. GORMAN, G. CHÁVEZ, P. GHYSELS, T. MARY, F.-H. ROUET, AND X. S. LI, Robust and accurate stopping criteria for adaptive randomized sampling in matrix-free hierarchically semiseparable construction, SIAM Journal on Scientific Computing, 41 (2019), pp. S61–S85.
- [22] L. GREENGARD AND V. ROKHLIN, A fast algorithm for particle simulations, J. Comp. Phys, 73 (1987), pp. 325–348.
- [23] L. GREENGARD AND V. ROKHLIN, A new version of the fast multipole method for the laplace equation in three dimensions, Acta numerica, 6 (1997), pp. 229–269.
- [24] W. HACKBUSCH, A sparse matrix arithmetic based on $\mathcal{H}$ -matrices. Part I: Introduction to

- $\mathcal{H}$ -matrices, Computing, 62 (1999), pp. 89–108.
- [25] W. HACKBUSCH, Hierarchical Matrices: Algorithms and Analysis, Springer Series in Computational Mathematics, 1st ed., 2015.
- [26] W. HACKBUSCH, B. KHOROMSKIJ, AND S. A. SAUTER, On H^2 -Matrices, in Lectures on Applied Mathematics, H.-J. Bungartz, R. H. W. Hoppe, and C. Zenger, eds., Berlin, Heidelberg, 2000, Springer Berlin Heidelberg, pp. 9–29.
- [27] N. HALKO, P.-G. MARTINSSON, AND J. A. TROPP, Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions, SIAM review, 53 (2011), pp. 217–288.
- [28] J. HARNAD AND F. BALOGH, Tau Functions and their Applications, Cambridge Monographs on Mathematical Physics, Cambridge University Press, 2021.
- [29] N. J. HIGHAM, Functions of Matrices: Theory and Computation, SIAM, 2008.
- [30] N. J. HIGHAM AND T. MARY, Solving block low-rank linear systems by LU factorization is numerically stable, IMA Journal of Numerical Analysis, 42 (2021), pp. 951–980.
- [31] K. L. HO AND L. GREENGARD, A fast direct solver for structured linear systems by recursive skeletonization, SIAM Journal on Scientific Computing, 34 (2012), pp. A2507–A2532.
- [32] A. IDA, H. NAKASHIMA, AND M. KAWAI, Parallel hierarchical matrices with block low-rank representation on distributed memory computer systems, in Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region, HPCAsia '18, New York, NY, USA, 2018, Association for Computing Machinery, p. 232–240.
- [33] C.-P. JEANNEROD, T. MARY, C. PERNET, AND D. S. ROCHE, Improving the complexity of block low-rank factorizations with fast matrix arithmetic, SIAM Journal on Matrix Analysis and Applications, 40 (2019), pp. 1478–1496.
- [34] W. B. JOHNSON AND J. LINDENSTRAUSS, Extensions of lipschitz mappings into hilbert space, Contemporary mathematics, 26 (1984), pp. 189–206, <https://api.semanticscholar.org/CorpusID:117819162>.
- [35] S. N. KARP, Moment curves and cyclic symmetry for positive grassmannians, Bulletin of the London Mathematical Society, 51 (2019), pp. 900–916.
- [36] J. LEVITT AND P.-G. MARTINSSON, Linear-complexity black-box randomized compression of hierarchically block separable matrices, arXiv preprint arXiv:2205.02990, (2022).
- [37] J. LEVITT AND P.-G. MARTINSSON, Randomized compression of rank-structured matrices accelerated with graph coloring, arXiv preprint arXiv:2205.03406, (2022).
- [38] L. LIN, J. LU, AND L. YING, Fast construction of hierarchical matrix representation from matrix-vector multiplication, Journal of Computational Physics, 230 (2011), pp. 4071–4087.
- [39] B. MARSH AND K. RIETSCH, The B-model connection and mirror symmetry for Grassmannians, Advances in Mathematics, 366 (2020), p. 107027.
- [40] P. MARTINSSON, Rapid factorization of structured matrices via randomized sampling, 2008. arXiv:0806.2339.
- [41] P. MARTINSSON AND V. ROKHLIN, A fast direct solver for boundary integral equations in two dimensions, J. Comp. Phys., 205 (2005), pp. 1–23.
- [42] P.-G. MARTINSSON, A fast randomized algorithm for computing a hierarchically semiseparable representation of a matrix, SIAM Journal on Matrix Analysis and Applications, 32 (2011), pp. 1251–1274.
- [43] P.-G. MARTINSSON, Compressing rank-structured matrices via randomized sampling, SIAM Journal on Scientific Computing, 38 (2016), pp. A1959–A1986.
- [44] P.-G. MARTINSSON, Fast direct solvers for elliptic PDEs, SIAM, 2019.
- [45] P.-G. MARTINSSON AND J. A. TROPP, Randomized numerical linear algebra: Foundations and algorithms, Acta Numerica, 29 (2020), pp. 403–572.
- [46] T. MARY, Block Low-Rank Multifrontal Solvers: Complexity, Performance, and Scalability, PhD thesis, Université de Toulouse, Toulouse, France, Nov. 2017.
- [47] E. MILLER AND B. STURMFELS, Combinatorial commutative algebra, in Graduate Texts in Mathematics, Springer New York, 2004.
- [48] V. MINDEN, K. L. HO, A. DAMLE, AND L. YING, A recursive skeletonization factorization based on strong admissibility, Multiscale Modeling & Simulation, 15 (2017), pp. 768–796.
- [49] C. MOU, Q. SONG, Y. ZHOU, A. DICKENSTEIN, B. EICK, K. BUZZARD, A. LEYKIN, AND Y. REN, DetGB: A Software Package for Computing Gröbner Bases of Determinantal Ideals, in Mathematical Software – ICMS 2024, Lecture Notes in Computer Science, Springer Nature Switzerland, Cham, 2024, pp. 354–364.
- [50] G. PICHON, E. DARVE, M. FAVERGE, P. RAMET, AND J. ROMAN, Sparse supernodal solver using block low-rank compression: Design, performance and analysis, Journal of Computational Science, 27 (2018), pp. 255–270.

- [51] J. S. SCOTT, Grassmannians and cluster algebras, Proceedings of the London Mathematical Society, 92 (2006), pp. 345–380.
- [52] M. SERGENT, D. GOUDIN, S. THIBAUT, AND O. AUMAGE, Controlling the memory subscription of distributed applications with a task-based runtime system, in 2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), 2016, pp. 318–327.
- [53] J. A. TROPP, A. YURTSEVER, M. UDELL, AND V. CEVHER, Practical sketching algorithms for low-rank matrix approximation, SIAM Journal on Matrix Analysis and Applications, 38 (2017), pp. 1454–1485.
- [54] J. XIA, S. CHANDRASEKARAN, M. GU, AND X. S. LI, Superfast multifrontal method for large structured linear systems of equations, SIAM Journal on Matrix Analysis and Applications, 31 (2010), pp. 1382–1411.
- [55] X. XING AND E. CHOW, Interpolative decomposition via proxy points for kernel matrices, SIAM Journal on Matrix Analysis and Applications, 41 (2020), pp. 221–243.
- [56] A. YESYPENKO, Randomized algorithms for the efficient solution of elliptic PDEs on modern architectures, PhD thesis, Oden Institute for Computational Engineering and Sciences, 2023.
- [57] A. YESYPENKO, C. CHEN, AND P.-G. MARTINSSON, A simplified fast multipole method based on strong recursive skeletonization, Journal of Computational Physics, (2024), p. 113707.
- [58] A. YESYPENKO AND P.-G. MARTINSSON, Randomized strong recursive skeletonization: Simultaneous compression and LU factorization of hierarchical matrices using matrix-vector products, arXiv:2311.01451 [math.NA], (2023).
- [59] A. YESYPENKO AND P.-G. MARTINSSON, SlabLU: a two-level sparse direct solver for elliptic PDEs, Advances in Computational Mathematics, 50 (2024), p. 90.